

# CO-I01 | 一条指令的一生

以 LOAD 追踪语义、硬件路径、快慢分支与精确提交

408 · 计算机组成原理综合专题

## 完整执行流程

ISA Semantic → Fetch / Decode → Datapath / Pipeline → EA → Translation / Cache / Memory → Result R

母指令取 `LOAD rd, disp(rs)`。它既需要一次指令访问，也需要一次数据访问；任一阶段可以变慢或异常，但只有全部前置条件成立，`rd` 才能被架构提交。

## 本手册的范围

本册只追踪一条指令跨越多个模块时的顺序、交接字段、快路径/慢路径和停止条件。位串、ISA、数据通路、流水线、存储、Cache、地址翻译和总线的具体定义仍以各自专题正文为准；本册不复制第二套定义。

## 1 术语先行：整条指令为什么需要交接

### 五个流程词

- **LOAD**：从有效地址读取指定宽度数据，并把扩展后的结果写入目标寄存器的指令。
- **交接字段**：一个阶段交给下一阶段的地址、数据、权限、状态和重试信息。
- **快路径**：取指、翻译、Cache 和执行都命中且没有冲突的常规路径。
- **慢路径**：遇到缺失、等待、异常或总线争用，需要暂停、修复或重试的路径。
- **精确提交**：发生异常时，较老指令的效果已经可见，出错指令和更年轻指令的禁止副作用仍不可见。

## 目录

|                                     |   |
|-------------------------------------|---|
| 1 术语先行：整条指令为什么需要交接                  | 1 |
| 2 第一步不是画硬件，而是写 LOAD 的架构状态差          | 2 |
| 3 完整轨迹包含两次访问                        | 2 |
| 3.1 Instruction fetch . . . . .     | 2 |
| 3.2 Decode 与 operand read . . . . . | 2 |
| 3.3 EA generation . . . . .         | 2 |

|    |                             |   |
|----|-----------------------------|---|
| 4  | 数据访问：翻译与 Cache 的双重身份证明      | 2 |
| 5  | Fast Path：所有局部预测都成功         | 3 |
| 6  | Slow Path：谁暂停，谁修复，哪里重试      | 3 |
| 7  | Ready、Forward 与 Commit 必须分开 | 3 |
| 8  | 性能只在成本边界正确时组合               | 4 |
| 9  | 母轨迹怎样迁移到其他指令                | 4 |
| 10 | 五列概念边界                      | 5 |
| 11 | 综合题调用协议                     | 5 |
| 12 | 各阶段的机制入口与停止条件               | 6 |

## 2 第一步不是画硬件，而是写 LOAD 的架构状态差

CO-02 给出

$$EA = Reg[rs] + SignExt(displacement), \quad value = M[EA, size],$$

$$Read = \{PC, Reg[rs], instruction\ bytes, M[EA]\}, \quad Write = \{Reg[rd], NextPC\},$$

并附 access size、signed/unsigned extension、alignment、privilege 和可能异常。CO-01 决定 displacement 怎样扩展、内存字节怎样解释并压入目标位宽。

CO-B01 把它标准化为

$$\{Read, Derived, Write, NextPC, Memory, Exception\},$$

再展开为 value dependency: PC 产生取指地址; 指令字产生字段;  $Reg[rs]$  与扩展立即数产生 EA; 翻译和 Cache/Memory 产生 load value; 只有无异常结果可进入  $Reg[rd]$ 。

## 3 完整轨迹包含两次访问

### 3.1 Instruction fetch

当前 PC 先按 ISA 对齐/权限规则形成 instruction access request。I-TLB/页表将 VA 翻成 PA, I-Cache 检查指令块副本; miss 时向下一级/主存取块。指令字 ready 后才可稳定译码。教学五级流水常把这一过程压缩为 IF 一拍, 题设若改变 TLB/Cache latency 就必须展开。

### 3.2 Decode 与 operand read

译码恢复 opcode、寄存器字段、displacement、访问宽度和符号扩展规则; 寄存器堆读取 base operand。非法指令、权限或字段约束可能在此形成异常, 不能把位串成功读取等同于语义合法。

### 3.3 EA generation

ALU 计算  $EA = base + extended\ displacement$ 。这里产生的是 virtual effective address; alignment/check 发生在哪一层由 ISA/实现给出。EA ready 不等于 load data ready, 也不等于可以写回。

## 4 数据访问：翻译与 Cache 的双重身份证明

CO-B02 接收  $(VA, access\ type, privilege, size)$ , 翻译输出  $(PA, permission, attributes, status)$ 。合法 PA 再进入 Cache 的 tag/index/offset 身份检查:

$$translation\ valid \wedge permission\ OK \wedge cache\ identity\ hit \Rightarrow data\ may\ return.$$

PIPT 通常先翻译后索引; VIPT 可用 page-offset 内的 index 与 TLB 并行, 但最终仍用 PA tag 和权限确认。TLB miss、Page Fault 与 Cache miss 必须分别路由。

5 Fast Path：所有局部预测都成功

在题设五级 in-order、I/D TLB hit、I/D Cache hit、无结构冲突和异常时，可写成：

| 阶段        | 主要结果                            | 负责的硬件层               | 交给下一步                    |
|-----------|---------------------------------|----------------------|--------------------------|
| IF        | instruction bytes ready         | CO04 + CO07/06       | instruction packet       |
| ID        | semantic/dependence packet      | CO02 + CO-B01        | operands/control intent  |
| EX        | EA ready                        | CO01/03              | data access request      |
| MEM       | PA valid, Cache hit, data ready | CO07 + CO-B02 + CO06 | typed load value         |
| WB/Commit | Integ[rd], NextPC 可见            | CO03/04              | next architectural state |

阶段名不是机制证明；题目若在 EX 前移/后移分支判断、让 memory 多周期或改变寄存器堆时序，必须用 need/ready 重新排拍。

6 Slow Path：谁暂停，谁修复，哪里重试

| 事件                    | 缺失/冲突                         | 处理者                     | 对指令的影响                           |
|-----------------------|-------------------------------|-------------------------|----------------------------------|
| I-TLB/I-Cache miss    | 指令翻译/块副本                      | 翻译或存储层次                 | IF 等待/重试                         |
| RAW hazard            | producer value 尚未到 need point | forwarding/interlock    | forward 或 stall                  |
| D-TLB miss            | 数据翻译副本                        | page walker/refill      | 数据访问重试                           |
| Page/permission fault | 映射/权限失败                       | 精确异常 + OS               | 年轻指令作废，修复后 restart 或终止           |
| D-Cache miss          | 数据块副本缺失                       | Cache/memory controller | victim/fill/retry, load value 延后 |
| Memory/bus busy       | 共享资源未获权                       | arbiter/controller      | transaction 等待，不等于 ISR           |

Page Fault 进入操作系统后超出本册的硬件流程；本册只保留 fault 的交接信息与重启边界。若访问 MMIO，地址属性可能绕过普通 Cache，由 I/O 控制器和总线事务接管；普通 DRAM fill 则按主存组织和访问成本完成。

7 Ready、Forward 与 Commit 必须分开

load value 从 Cache/memory 返回是 **ready**；旁路可把它交给年轻 consumer，是 **forwardable**；只有该 LOAD 成为允许改变软件可见状态的指令时才 **commit**。经典 load-use hazard 来自 consumer need

早于 load ready，而不是寄存器名字相同本身。

精确异常要求：发生 fault 的 LOAD 不写  $Reg[rd]$ ，更年轻指令不留下架构副作用，更老指令已按模型完成；修复后从 ISA 规定的 restart point 重新执行。内部可曾做过推测访问，不等于软件可见提交。

三条常见错误链

EA 已算出  $\Rightarrow$  数据已取回；TLB miss  $\Rightarrow$  Page Fault；Cache 返回数据  $\Rightarrow$  LOAD 已提交。三条都跳过了中间 handoff 与状态条件。

8 性能只在成本边界正确时组合

总 CPU time 仍是  $IC \times CPI \times T_{clk}$ 。单条 LOAD 的额外等待可能来自 hazard、TLB、Cache、memory 和 bus，但不能把已包含在 miss penalty 中的下一级 AMAT 再加一次，也不能同时把可并行的 TLB/Cache latency 串行求和。

先写事件树和条件概率，再根据题设重叠关系组合。例如近似数据访问成本可写

$$T_D = T_{TLB/cache\ fast} + P(TLB\ miss)T_{walk,extra} + P(Cache\ miss \mid translation\ succeeds)T_{fill,extra},$$

其中 page fault 通常应作为独立稀有 slow path，且其 OS/磁盘成本不能混入普通 Cache miss。

9 母轨迹怎样迁移到其他指令

| 指令           | 保留                          | 删除/替换                   | 提交条件              |
|--------------|-----------------------------|-------------------------|-------------------|
| ADD          | fetch/decode/execute/timing | 无数据翻译与 Cache；ALU 产值     | 无异常且结果到 commit    |
| STORE        | fetch/EA/translation/Cache  | 无 $Reg[rd]$ 写回；写缓冲/策略接管 | 内存副作用按架构顺序可见      |
| BRANCH       | fetch/decode/compare/NextPC | 无 data load；加入预测/flush  | 正确路径 PC 与年轻指令状态一致 |
| MMIO<br>LOAD | fetch/EA/translation        | 普通 Cache 路径可能被属性旁路      | 控制器事务合法完成且无 fault |

迁移方法是删改 handoff，不是重画一套全新 CPU。若某指令要求新数据源、目的或异常边界，先回 CO-02/B01 修改语义包，再判断 CO-03 通路是否支持。

10 五列概念边界

| 概念 A      | ≠ 概念 B             | 真正区别与题面信号           | 混淆后果      |
|-----------|--------------------|---------------------|-----------|
| EA ready  | ≠ load value ready | 地址已生成 vs 数据已返回      | 漏翻译/Cache |
| TLB miss  | ≠ Page Fault       | 翻译副本缺失 vs 映射/权限失败   | 错写 OS 介入  |
| Cache hit | ≠ permission OK    | 数据副本存在 vs 访问合法      | 非法数据提交    |
| forward   | ≠ commit           | 内部传值 vs 架构状态可见      | 精确异常破坏    |
| bus wait  | ≠ interrupt        | 共享事务等待 vs CPU 控制流事件 | 把停顿写成保存现场 |
| 综合流程      | ≠ 单一专题             | 跨模块过程 vs 局部机制定义     | 产生第二套定义   |

11 综合题调用协议

- 1. 写 ISA state delta 与异常集合；
- 2. 把 instruction fetch 与 data access 分开；
- 3. 逐值标 producer、ready、consumer need 和 commit；
- 4. 对 VA 访问写 translation result，再做 Cache identity；
- 5. 每个 miss/fault/冲突写检测者、处理者、重试点；
- 6. 标共享资源、允许重叠项和成本是否已包含；
- 7. 在无异常路径证明  $Reg[rd]$  与 NextPC 的精确提交；
- 8. 局部推导不足时回到对应专题的定义，不在本册补第二套定义。

压缩成第一动作

看到“分析一条指令全过程”，先画三条线：值依赖线、时间 ready/need 线、异常/提交线；再把每个节点归入对应专题。

12 各阶段的机制入口与停止条件

| 问题信号                          | 进入的专题      | 本综合流程的停止点                   |
|-------------------------------|------------|-----------------------------|
| 位宽 / 扩展舍入                     | CO-01      | 得到 typed finite-width value |
| 编码 / EA异常语义                   | CO-02      | 得到 state delta packet       |
| MUX / 微操作控制字                  | CO-03      | 得到单指令路径与控制                  |
| hazard / stall flush / commit | CO-04      | 得到 need-ready 与提交顺序         |
| DRAM / 模块介质成本                 | CO-05      | 得到访问服务模型                    |
| tag / miss write policy       | CO-06      | 得到副本状态转换                    |
| TLB / PTE page walk           | CO-07      | 得到 translation result       |
| bus / MMIO DMA                | CO-08      | 得到 transaction completion   |
| 两侧 handoff                    | CO-B01/B02 | 得到标准接口包                     |

一句话

一条 LOAD 的一生不是固定五阶段口诀，而是“语义承诺沿值依赖和时间路径逐层获得证据，任何 slow path 都必须回到明确重试点，最终只在无异常且顺序合法时提交”。